TechTransit — Backlog de tareas para desarrolladores junior

> **Versión:** 1.0 (4-nov-2026) · **Fuente:** comparación entre el documento de alcance v1.0 y el código actual de `main`.

Este documento lista lo que **falta** en el sistema, partido en tareas pequeñas que puede tomar una persona que está empezando. Cada tarea trae su **historia de usuario**, el **contexto** (qué ya existe y en qué archivos mirar), una **guía técnica** y los **criterios de aceptación** con los que se revisa.

Cómo usar este backlog

1. **Antes de la primera tarea**, lee el README y levanta el proyecto con la guía de desarrollo. Si no puedes hacerlo funcionar en tu máquina, esa es tu primera tarea: pide ayuda.
2. Toma las tareas **en orden de bloque**. Dentro de un bloque puedes elegir cualquiera que no tenga dependencias pendientes.
3. Una tarea = una rama = un Pull Request (`feat/T-07-historial-proceso`, `fix/T-04-register`...).
4. Si una tarea dice 🟠 **Requiere decisión**, no empieces hasta que el líder confirme la opción.

Niveles y tamaños

Nivel	Significado
🟢 **Junior**	Cambios en 1–3 archivos siguiendo un patrón que ya existe en el código. Ideal para empezar.
🔵 **Junior+**	Toca backend **y** frontend, o requiere una migración de base de datos. Conviene tener un par de tareas 🟢 hechas antes.

Tamaño	Tiempo orientativo
S	Hasta medio día
M	1 a 2 días
L	3 a 5 días

Definición de "Hecho" (aplica a TODAS las tareas)

Además de sus criterios propios, una tarea solo está terminada si:

- ☐ Está en una rama propia con commits del estilo `feat(modulo): ...` / `fix(modulo): ...`.
- ☐ `npm run lint` y `npm run build` pasan sin errores ni advertencias.
- ☐ Se probó a mano en el navegador siguiendo los criterios de aceptación, con la BD local (`npm run db:up`).

- ☐ Si cambió el `schema.prisma`, la migración nueva está incluida en el commit.
- ☐ El Pull Request explica **qué** cambió y **cómo probarlo**, e incluye capturas si hay cambios visuales.
- ☐ No hay `console.log` olvidados, código comentado ni secretos (tokens, contraseñas reales).
- ☐ Las fechas de calendario se formatean con `lib/fechas.ts` (ver README §7.4), nunca con `new Date()` directo.
- ☐ Si la tarea cambia algo descrito en el README o en `DESARROLLO.md`, se actualizó la documentación.

Resumen

ID	Tarea	Bloque	Nivel	Tamaño	Depende de
T-01	Formatear el código con Prettier	0 · Higiene	🟢	S	—
T-02	Eliminar restos de Render y Supabase	0 · Higiene	🟢	S	—
T-03	Inventario de licencias (`LICENSES.md`)	0 · Higiene	🟢	S	—
T-04	Cerrar el registro público de usuarios	1 · Seguridad	🟢	S	—
T-05	Textos del semáforo según días restantes	2 · Funcionalidad	🟢	S	🟠 decisión
T-06	Días restantes en las tarjetas del Kanban	2 · Funcionalidad	🟢	S	—
T-07	Ver el historial de un proceso	2 · Funcionalidad	🟢	M	—
T-08	Filtros por entidad y responsable en Precontractual	2 · Funcionalidad	🟢	M	—
T-09	Recordar notificaciones leídas	2 · Funcionalidad	🟢	S	—
T-10	Exportar procesos a CSV (Excel)	2 · Funcionalidad	🟢	M	T-08 (opcional)
T-11	Máximo 3 hitos por contrato	2 · Funcionalidad	🟢	S	🟠 decisión
T-12	Asignar aliados a un proceso	2 · Funcionalidad	🟢	M	—
T-13	Gestión de usuarios en Catálogos	2 · Funcionalidad	🔵	L	T-04
T-14	Cambiar mi contraseña	2 · Funcionalidad	🔵	M	—
T-15	Configurar pruebas automáticas (Vitest)	3 · Calidad	🔵	M	—
T-16	Campos faltantes del proceso: canal, prioridad y enlace	4 · Modelo de datos	🔵	L	T-08
T-17	Ciclo de vida del contrato	4 · Modelo de datos	🔵	L	🟠 decisión

Bloque 0 — Higiene del repositorio

Tareas cortas, ideales para el primer día: obligan a recorrer el proyecto sin riesgo de romper funcionalidad.

T-01 · Formatear el código con Prettier

🟢 Junior · **S** · Sin dependencias

Historia de usuario

> Como **desarrollador del equipo**, quiero que todo el código tenga el mismo formato, para que los Pull Requests muestren solo cambios reales y no diferencias de espacios.

Contexto

- `npm run format:check` reporta hoy **34 archivos** sin formatear.
- La configuración ya existe en `.prettierrc`.

Guía técnica

1. Ejecuta `npm run format`.
2. Revisa el diff: solo deben cambiar espacios, comillas y saltos de línea, nunca la lógica.
3. Haz un commit **que solo contenga el formateo**: `style: format codebase with prettier`.

Criterios de aceptación

- ☐ `npm run format:check` termina con *"All matched files use Prettier code style!"*.
- ☐ `npm run lint` y `npm run build` siguen pasando.
- ☐ El PR no mezcla otros cambios con el formateo.

T-02 · Eliminar restos de Render y Supabase

🟢 Junior · **S** · Sin dependencias

Historia de usuario

> Como **desarrollador nuevo**, quiero que el repositorio no tenga configuraciones de servicios que ya no usamos, para no confundirme sobre dónde y cómo se despliega la aplicación.

Contexto

- El despliegue se hará en un servidor **on-premise**; Render y Supabase se descartaron.
- Quedan restos en: `render.yaml`, `backend/.env.example` (URLs de Supabase), el comentario de `frontend/src/lib/api.ts` (menciona Render) y la sección 14 del `README.md`.

Guía técnica

1. Busca referencias: `grep -rni "render\|supabase" --exclude-dir=node_modules .`

2. Borra `render.yaml`.
3. En `backend/.env.example` reemplaza las URLs de Supabase por una URL genérica de PostgreSQL (`postgresql://USUARIO:CONTRASEÑA@HOST:5432/techtransit`).
4. Actualiza los comentarios y la documentación que mencionen Render o Supabase.

Criterios de aceptación

- ☐ `render.yaml` ya no existe.
- ☐ El `grep` del paso 1 no encuentra referencias, salvo en `docs/TechTransit_Alcance_v1.0.docx` (documento histórico).
- ☐ `backend/.env.example` sigue teniendo todas las variables que valida `backend/src/config/env.ts`.
- ☐ La guía `DESARROLLO.md` y el README siguen funcionando paso a paso (pruébalo).

T-03 · Inventario de licencias (`LICENSES.md`)

🟢 Junior · **S** · Sin dependencias

Historia de usuario

> Como **responsable del producto**, quiero un listado de las licencias de todas las librerías que usamos, para asegurar que ninguna nos obligue a publicar el código (el repositorio es privado).

Contexto

- El alcance (§9, decisión 6) exige solo licencias permisivas (MIT, Apache-2.0, BSD, ISC o similares) y prohíbe GPL/LGPL/AGPL. También pide documentarlo en `LICENSES.md`.

Guía técnica

1. Genera el resumen: `npx license-checker-rseidelsohn --production --summary` y el detalle con `--csv` o `--markdown`.
2. Crea `LICENSES.md` en la raíz con: fecha de generación, resumen por tipo de licencia y la tabla de dependencias de producción (nombre, versión, licencia).
3. Añade al README, en "Scripts disponibles", el comando para regenerarlo.

Criterios de aceptación

- ☐ Existe `LICENSES.md` con resumen y tabla de dependencias de producción.
- ☐ Si alguna dependencia es GPL, LGPL, AGPL o "UNKNOWN", está **resaltada** y se avisó al líder en el PR.
- ☐ El documento explica cómo regenerarlo.

Bloque 1 — Seguridad

T-04 · Cerrar el registro público de usuarios

🟢 Junior · **S** · Sin dependencias · **Prioridad alta: hacer antes de exponer el servidor**

Historia de usuario

> Como **administrador**, quiero que solo yo pueda crear cuentas, para que nadie ajeno a la empresa pueda registrarse y ver información de los contratos.

Contexto

- `POST /api/v1/auth/register` es público: cualquiera que llegue al API puede crearse una cuenta COLABORADOR.
- El frontend **no usa** este endpoint, y los administradores ya crean usuarios con `POST /api/v1/usuarios` (`backend/src/modules/usuarios/`, protegido con `requireRoles('ADMIN')`).

Guía técnica

1. En `backend/src/modules/auth/auth.router.ts` elimina la ruta `/register`.
2. En `auth.service.ts` elimina la función `register` y en `auth.schemas.ts` el `registerSchema` (y su tipo `RegisterDto`), si ya no se usan.
3. Quita la fila de `/auth/register` de la tabla de la API en el README (§8) y el pendiente de la sección 14.

Criterios de aceptación

- ☐ `POST /api/v1/auth/register` responde **404**.
- ☐ Login, refresh y logout siguen funcionando (inicia y cierra sesión en el navegador).
- ☐ Un ADMIN puede seguir creando usuarios con `POST /api/v1/usuarios`.
- ☐ No queda código sin uso (`npm run lint` y `npm run build` pasan).

Bloque 2 — Funcionalidad

T-05 · Textos del semáforo según días restantes

🟢 Junior · **S** · 🟠 **Requiere decisión**

Historia de usuario

> Como **director de proyecto**, quiero que el semáforo de un contrato diga si está vencido o solo en riesgo, para no alarmarme con un contrato que todavía tiene días.

Contexto

- Hoy `frontend/src/components/ui/SemaforoBadge.tsx` muestra ROJO siempre como **"Vencido"**, aunque al contrato le queden días (el semáforo se pone ROJO con ≤ 15 días restantes).
- El alcance (Módulo 3) define: **VENCIDO** (< 0 días) · **CRÍTICO** (≤ 15) · **ATENCIÓN** · **EN CURSO**.

🟠 **Decisión necesaria:** confirmar los textos. Propuesta:

Semáforo	Condición	Texto
ROJO	`fechaFin` ya pasó	**Vencido**
ROJO	aún no vence	**Crítico**
AMARILLO	—	**Atención**
VERDE	—	**En curso**

Guía técnica

1. Haz que `SemaforoBadge` reciba también la `fechaFin` (opcional) y use `diasHasta()` de `lib/fechas.ts`.
2. Actualiza todos los lugares donde se usa el componente (búscalo con `grep -rn SemaforoBadge frontend/src`). En el dashboard se usa como leyenda de conteos (sin fecha): ahí ROJO debe decir **"Crítico / Vencido"**.
3. Revisa que el filtro "Todos los semáforos" de Contratos use los mismos textos.

Criterios de aceptación

- ☐ Un contrato ROJO con fecha de fin futura muestra **"Crítico"**.
- ☐ Un contrato ROJO con fecha de fin pasada muestra **"Vencido"**.
- ☐ AMARILLO muestra "Atención" y VERDE "En curso", en la lista, el detalle y el dashboard.
- ☐ Los colores no cambian.

T-06 · Días restantes en las tarjetas del Kanban

🟢 Junior · **S** · Sin dependencias

Historia de usuario

> Como **analista**, quiero ver en cada tarjeta del Kanban cuántos días faltan para el cierre, con un color de alerta, para priorizar las licitaciones más urgentes sin abrir cada una.

Contexto

- Requisito RF-13 del alcance: alerta visual **rojo ≤ 5 días**, **ámbar 6–15 días**.
- La tarjeta está en `frontend/src/pages/PrecontractualPage.tsx` y hoy solo muestra "Cierre: ".
- Ya existe `diasHasta()` en `frontend/src/lib/fechas.ts`. `ContratosPage.tsx` hace algo parecido con "días restantes" y te sirve de ejemplo.

Guía técnica

1. Junto a la fecha de cierre, muestra `"Cierra en N días"`, `"Cierra hoy"` o `"Cerró hace N días"`.
2. Color: rojo (`var(--rojo)`) si quedan 0–5 días, ámbar (`var(--amarillo)`) entre 6 y 15, y el color normal en otro caso.
3. Solo aplica a los estados ESTRUCTURACION y PRESENTACION. En GANADO, NO_ADJUDICADO y DESIERTO no se muestra el contador.

Criterios de aceptación

- ☐ Un proceso que cierra hoy dice "Cierra hoy" en rojo.
- ☐ Uno que cierra en 10 días dice "Cierra en 10 días" en ámbar.
- ☐ Uno que cierra en 30 días se muestra sin color de alerta.
- ☐ Uno sin fecha de cierre no muestra contador ni produce errores.
- ☐ Los procesos en estados finales no muestran el contador.

T-07 · Ver el historial de un proceso

🟢 Junior · **M** · Sin dependencias

Historia de usuario

> Como **director**, quiero ver quién movió un proceso de estado, cuándo y con qué comentario, para tener trazabilidad sin preguntar al equipo.

Contexto

- Requisito RF-16. El backend **ya lo entrega**: `GET /api/v1/procesos/:id` incluye `historial` (últimos 20 cambios, con `usuario.nombre`, `estadoAnterior`, `estadoNuevo`, `comentario` y `createdAt`).
- El frontend no lo muestra en ningún lado.

Guía técnica

1. En `frontend/src/lib/api.procesos.ts` agrega el tipo `HistorialProceso` y una función `get(id)` si no existe.
2. En `PrecontractualPage.tsx` agrega en cada tarjeta un botón "Historial" que abra un modal (reutiliza el patrón de `ModalOverlay` / `ModalShell` de `CatalogosPage.tsx`).
3. Muestra una línea de tiempo: fecha y hora, usuario, `estadoAnterior → estadoNuevo` y el comentario.
4. `createdAt` es un instante real (no una fecha de calendario), así que aquí sí se usa `new Date(createdAt).toLocaleString('es-CO')`.

Criterios de aceptación

- ☐ Al mover un proceso de ESTRUCTURACION a PRESENTACION, el cambio aparece **arriba** del historial con tu nombre.
- ☐ Se ven la fecha, la hora local, el usuario, los estados y el comentario (si lo hay).
- ☐ Un proceso recién creado muestra el registro de su creación (el backend lo guarda con `estadoAnterior` vacío; muéstralo como "Creado en ESTRUCTURACION").

- ☐ Mientras carga se ve un indicador, y si la petición falla se muestra un mensaje de error.
- ☐ Un usuario COLABORADOR también puede ver el historial (es solo lectura).

T-08 · Filtros por entidad y responsable en Precontractual

🟢 Junior · **M** · Sin dependencias

Historia de usuario

> Como **director**, quiero filtrar el Kanban por entidad y por responsable, para revisar rápidamente los procesos de un cliente o de una persona del equipo.

Contexto

- Requisito RF-14 (parcial). Hoy solo existe el filtro por estado (`filterEstado` en `PrecontractualPage.tsx`).
- El backend **ya acepta** `entidadId` y `responsableId` en `GET /api/v1/procesos` (ver `listProcesosSchema` en `backend/src/modules/procesos/procesos.schemas.ts`).
- Ya existe un componente `Select` en `frontend/src/components/ui/Select.tsx` .

Guía técnica

1. Carga la lista de entidades (`entidadesApi`) y de usuarios para llenar los selects. ⚠ `GET /usuarios` es solo para ADMIN; para los demás roles, arma la lista de responsables a partir de los procesos ya cargados (sin duplicados), o consulta al líder.
2. Agrega dos `Select` : "Todas las entidades" y "Todos los responsables".
3. Envía los filtros elegidos como parámetros a `procesosApi.list(...)` .
4. Agrega un botón "Limpiar filtros".

Criterios de aceptación

- ☐ Al elegir una entidad, el Kanban muestra solo sus procesos y el contador "N procesos en total" se actualiza.
- ☐ Los filtros se combinan entre sí (entidad + responsable + estado).
- ☐ "Limpiar filtros" vuelve a mostrar todo.
- ☐ Funciona con los tres roles (prueba con los tres usuarios del seed).

T-09 · Recordar notificaciones leídas

🟢 Junior · **S** · Sin dependencias

Historia de usuario

> Como **usuario**, quiero que las notificaciones que marqué como leídas sigan así al recargar la página, para no revisar las mismas alertas una y otra vez.

Contexto

- En `frontend/src/pages/NotificacionesPage.tsx` las leídas se guardan en un `useState` (`leidas`), así que se pierden al recargar.
- Las notificaciones se calculan en el backend y cada una tiene un `id` estable (por ejemplo `contrato-rojo-<uuid>`).
- `frontend/src/lib/tokenStorage.ts` es un buen ejemplo de cómo usar `localStorage`.

Guía técnica

1. Crea `frontend/src/lib/notificacionesLeidas.ts` con `get(userId)` y `marcar(userId, ids)`, guardando en `localStorage` bajo la clave `tt_leidas_<userId>` (el id del usuario está en `useAuthStore`).
2. Envuelve el acceso a `localStorage` en `try/catch` (puede fallar en modo privado).
3. Usa ese módulo en `NotificacionesPage` para leer el estado inicial y para guardar cada cambio.
4. El contador de la campana está en `frontend/src/layouts/AppLayout.tsx` y usa `GET /notificaciones/conteo`, que devuelve el total sin saber cuáles se leyeron. Cámbialo para usar `notificacionesApi.list()` y contar solo las que no están en las leídas (son pocas, así que no afecta el rendimiento).
5. **Limpieza:** al cargar, descarta los ids guardados que ya no estén en la lista actual.

Criterios de aceptación

- ☐ Marcar una alerta como leída y recargar (F5): sigue leída.
- ☐ "Marcar todas como leídas" y recargar: todas siguen leídas y la campana muestra 0.
- ☐ Al iniciar sesión con otro usuario en el mismo navegador, ese usuario ve sus propias leídas.
- ☐ Si aparece una alerta nueva, sale como no leída.

T-10 · Exportar procesos a CSV (Excel)

🟢 Junior · **M** · Opcional: después de T-08

Historia de usuario

> Como **director**, quiero descargar el listado de procesos en un archivo que abra en Excel, para compartir reportes con la gerencia.

Contexto

- Requisito RF-17 (prioridad baja).
- **No agregues librerías de Excel**: un CSV bien armado abre directamente en Excel y no trae problemas de licencias (ver T-03).

Guía técnica

1. Crea `frontend/src/lib/exportarCsv.ts` con una función genérica `descargarCsv(nombreArchivo, columnas, filas)`.
2. Usa **punto y coma** (`;`) como separador (Excel en español lo espera así) y antepón el BOM `` para que las tildes y la ñ se vean bien.
3. Escapa los valores: si contienen `;`, `"` o saltos de línea, enciérralos entre comillas y duplica las comillas internas.

4. Agrega en Precontractual un botón "Exportar" que descargue **los procesos visibles con los filtros activos**. Columnas: número de proceso, nombre, entidad, estado, responsable, valor estimado, fecha de cierre y aliados.
5. Fechas en formato `YYYY-MM-DD` y valores numéricos sin separador de miles.
6. Nombre del archivo: `procesos_YYYY-MM-DD.csv`.

Criterios de aceptación

- ☐ El archivo abre en Excel con cada dato en su columna y las tildes y la ñ correctas.
- ☐ Un nombre con punto y coma o comillas no desordena las columnas.
- ☐ Si hay filtros activos, solo se exportan los procesos filtrados.
- ☐ Con 0 procesos, el botón está deshabilitado o descarga solo los encabezados.

T-11 · Máximo 3 hitos por contrato

🟢 Junior · **S** · 🟠 **Requiere decisión**

Historia de usuario

> Como **director**, quiero que cada contrato tenga como máximo 3 hitos, para mantener el seguimiento simple, como en el formato de Excel que usamos hoy.

Contexto

- Requisito RF-22 del alcance. Hoy no hay límite.

🟠 **Decisión necesaria:** confirmar si el límite de 3 sigue vigente, o si se elimina este requisito.

Guía técnica

1. Backend: en `createHito` (`backend/src/modules/contratos/contratos.service.ts`) cuenta los hitos del contrato y, si ya hay 3, lanza `createError('Un contrato admite máximo 3 hitos', 400)`. Define el límite como constante (`MAX_HITOS = 3`).
2. Frontend: en `ContratosPage.tsx` deshabilita el botón "Agregar hito" cuando haya 3 y muestra un aviso ("Máximo 3 hitos").

Criterios de aceptación

- ☐ Con 3 hitos, el botón está deshabilitado y se ve el aviso.
- ☐ Un `POST /contratos/:id/hitos` directo (por ejemplo con `curl`) con 3 hitos ya creados responde **400** con el mensaje.
- ☐ Al borrar un hito, se puede volver a agregar.

T-12 · Asignar aliados a un proceso

🟢 Junior · **M** · Sin dependencias

Historia de usuario

> Como **analista**, quiero elegir las empresas aliadas al crear o editar un proceso, para registrar con quién nos presentamos a cada licitación.

Contexto

- Requisito RF-15. Las tarjetas ya **muestran** los aliados, pero el formulario no permite elegirlos.
- El backend **ya está listo**: acepta `aliadoIds: string[]` al crear y al editar. Al editar reemplaza la lista completa, pero solo si envías `aliadoIds`; si lo omites, deja los aliados como estaban. Es una tarea **solo de frontend**.

Guía técnica

1. En el modal de proceso (`PrecontractualPage.tsx`), carga los aliados activos (`aliadosApi.list`).
2. Agrega una selección múltiple: casillas con buscador o "chips" que se agregan y se quitan.
3. Al editar, precarga los aliados actuales (`proceso.aliados[].aliadoId`).
4. Envía `aliadoIds` en el payload, también cuando la lista quede vacía (`[]`), para que se puedan quitar todos.

Criterios de aceptación

- ☐ Crear un proceso con 2 aliados: la tarjeta los muestra.
- ☐ Editarlo, quitar uno y agregar otro: la tarjeta refleja exactamente la nueva lista.
- ☐ Editar otros campos sin tocar los aliados no los borra.
- ☐ Quitar todos los aliados de un proceso funciona.
- ☐ Solo aparecen aliados activos para elegir.

> **Extensión futura (no incluida):** rol del aliado (líder, integrante...) y % de participación. La tabla `proceso_aliado` ya tiene las columnas `rol_consorcio` y `porcentaje_participacion`.

T-13 · Gestión de usuarios en Catálogos

🔵 Junior+ · **L** · Depende de T-04

Historia de usuario

> Como **administrador**, quiero crear, editar y desactivar usuarios desde la aplicación, para no depender de un desarrollador cada vez que entra o sale alguien del equipo.

Contexto

- Requisito RF-03. El backend **ya existe**: `GET/POST/PUT/DELETE /api/v1/usuarios` (solo ADMIN; `DELETE` desactiva). Revisa `usuarios.schemas.ts` para ver los campos.
- `CatalogosPage.tsx` tiene pestañas Entidades y Aliados (`type Tab = 'entidades' | 'aliados'`). `EntidadesTab` es el mejor ejemplo a copiar.

Guía técnica

1. Crea los tipos y la interfaz `usuariosApi` en `frontend/src/lib/api.catalogos.ts`.
2. Agrega una pestaña **Usuarios**, visible **solo si** `user.rol === 'ADMIN'` (`useAuthStore`).
3. Tabla con nombre, email, rol y estado (activo o inactivo), más un filtro "Mostrar inactivos".
4. Modal de crear y editar: nombre, email, rol (select) y contraseña (obligatoria al crear, opcional al editar).
5. Botón de desactivar con confirmación.
6. Un admin **no puede desactivarse a sí mismo**: ocúltalo en la UI y valídalo también en el backend (`createError(..., 400)`).

Criterios de aceptación

- ☐ SUPERVISOR y COLABORADOR **no ven** la pestaña Usuarios.
- ☐ El ADMIN crea un usuario y ese usuario puede iniciar sesión con la contraseña asignada.
- ☐ Cambiar el rol de un usuario se refleja en sus permisos al volver a iniciar sesión.
- ☐ Un usuario desactivado no puede iniciar sesión (responde "Credenciales inválidas").
- ☐ El ADMIN no puede desactivarse a sí mismo (ni desde la UI ni llamando al API directamente).
- ☐ Un email duplicado muestra un error claro en el formulario.
- ☐ La contraseña nunca aparece en ninguna respuesta del API.

T-14 · Cambiar mi contraseña

🔵 Junior+ · **M** · Sin dependencias

Historia de usuario

> Como **usuario**, quiero cambiar mi contraseña desde la aplicación, para reemplazar la contraseña temporal que me asignó el administrador.

Contexto

- No existe pantalla de perfil ni endpoint para esto.
- La contraseña se guarda con `bcrypt` (ver `auth.service.ts`: `bcrypt.hash(..., 12)` y `bcrypt.compare`).

Guía técnica

1. **Backend:** nuevo endpoint `PATCH /api/v1/auth/password`, protegido con `requireAuth`.
   - Schema Zod: `{ actual: string, nueva: string (mín. 8 caracteres) }`.
   - Verifica `actual` con `bcrypt.compare`; si no coincide, responde **400** "La contraseña actual no es correcta".
   - Guarda el hash de `nueva` y **borra el** `refreshToken` del usuario (cierra las demás sesiones).
   - Responde **204**.
2. **Frontend:** en el menú del usuario (avatar arriba a la derecha o la tarjeta de la barra lateral) agrega "Cambiar contraseña", que abre un modal con: actual, nueva y confirmar nueva.
3. Valida en el formulario que "nueva" y "confirmar" coincidan antes de enviar.
4. Al terminar, muestra un mensaje de éxito y cierra la sesión para que el usuario entre con la contraseña nueva.

Criterios de aceptación

- ☐ Con la contraseña actual correcta, el cambio funciona y la nueva sirve para iniciar sesión.
- ☐ La contraseña anterior deja de funcionar.
- ☐ Con la contraseña actual incorrecta, se muestra el error y no cambia nada.
- ☐ Una contraseña nueva de menos de 8 caracteres, o que no coincide con la confirmación, se rechaza con un mensaje claro.
- ☐ Sin token, el endpoint responde **401**.

Bloque 3 — Calidad

T-15 · Configurar pruebas automáticas (Vitest)

🔵 Junior+ · **M** · Sin dependencias

Historia de usuario

> Como **equipo**, queremos pruebas automáticas de las reglas de negocio, para detectar errores antes de llegar a producción y acercarnos a la meta de cobertura del alcance (≥ 70 %).

Contexto

- `backend/package.json` tiene `"test": "jest"`, pero Jest no está instalado y no hay pruebas.
- Se recomienda **Vitest**: funciona con TypeScript sin configuración extra.
- Las funciones más importantes para empezar son puras (no tocan la BD):
  - `backend/src/lib/fechas.ts`: `hoyFecha`, `diasEntre`, `diasHasta`.
  - `calcularSemaforo` en `backend/src/modules/contratos/contratos.service.ts`.
  - `validarTransicion` en `backend/src/modules/procesos/procesos.service.ts` (hay que exportarla).

Guía técnica

1. `npm install -D -w backend vitest @vitest/coverage-v8`.
2. Cambia el script por `"test": "vitest run"` y agrega `"test:watch": "vitest"` y `"test:coverage": "vitest run --coverage"`.
3. Crea las pruebas junto al código (`fechas.test.ts`, `contratos.service.test.ts`...).
4. Para "hoy", usa `vi.useFakeTimers()` y `vi.setSystemTime(...)` y así las pruebas no dependen de la fecha real.
5. Excluye los `*.test.ts` del build en `backend/tsconfig.json`.
6. Agrega un paso `npm test -w backend` al CI (`.github/workflows/ci.yml`). ⚠ Para subir cambios a esa carpeta tu token de GitHub necesita el permiso `workflow`.

Casos mínimos a cubrir

Función	Casos
`diasHasta`	hoy = 0 · mañana = 1 · ayer = −1
`hoyFecha`	a las 23:30 hora de Colombia (04:30 UTC del día siguiente) devuelve la fecha de **Colombia**
`calcularSemaforo`	terminado a tiempo → VERDE · vencido → ROJO · ≤ 15 días → ROJO · brecha de 10–20 puntos → AMARILLO · al día → VERDE
`validarTransicion`	ESTRUCTURACION→PRESENTACION permitido · ESTRUCTURACION→GANADO error · GANADO→cualquier estado error

Criterios de aceptación

- ☐ `npm test -w backend` corre y pasa todos los casos de la tabla.
- ☐ Si se cambia a propósito una regla (por ejemplo 15 días → 10), alguna prueba falla.
- ☐ `npm run build` no incluye los archivos de prueba en `backend/dist`.
- ☐ El CI ejecuta las pruebas.
- ☐ El README explica cómo correr las pruebas.

Bloque 4 — Modelo de datos

Estas tareas cambian la base de datos. Antes de empezar, repasa el flujo de migraciones en `DESARROLLO.md` §5 y la guía "agregar un módulo" del README §9.

T-16 · Campos faltantes del proceso: canal, prioridad y enlace

🔵 Junior+ · **L** · Depende de T-08

Historia de usuario

> Como **analista**, quiero registrar por qué canal se publicó el proceso, su prioridad y el enlace a la publicación, para tener toda la información que hoy guardamos en el Excel y filtrar por ella.

Contexto

- El alcance (tabla `proceso_precontractual`) incluye campos que el modelo actual no tiene:
  - `canal`: SECOP_I · SECOP_II · BOLSA_MERCANTIL · ACUERDO_MARCO · CONTRATACION_DIRECTA · REGIMEN_ESPECIAL · OTRO
  - `prioridad`: CRITICA · ALTA · MEDIA · NORMAL
  - `linkPublicacion`: URL del proceso en SECOP u otro portal
- Los tres son **opcionales**, para no romper los procesos existentes.

Guía técnica

1. `schema.prisma`: crea los enums `CanalProceso` y `PrioridadProceso` y agrega los 3 campos opcionales a `ProcesoPrecontractual` (con `@map` en snake_case).
2. Migración: `npm run db:migrate -w backend -- --name proceso_canal_prioridad_link`.

3. `procesos.schemas.ts`: agrega los campos (`z.enum([...])`, `z.url()`), y en `listProcesosSchema` los filtros `canal` y `prioridad`.
4. `procesos.service.ts`: aplica los filtros nuevos en `listProcesos`.
5. Frontend: tipos en `api.procesos.ts`, campos en el formulario, un indicador de prioridad en la tarjeta, el enlace "Ver publicación" (abre en una pestaña nueva con `rel="noopener noreferrer"`) y los filtros nuevos junto a los de T-08.

Criterios de aceptación

- ☐ La migración corre sobre una BD con datos y los procesos existentes siguen intactos (campos en `NULL`).
- ☐ Se puede crear y editar un proceso con canal, prioridad y enlace.
- ☐ Un enlace que no es una URL válida responde **400** con el error en el campo.
- ☐ Filtrar por canal y por prioridad funciona, también combinado con los demás filtros.
- ☐ El enlace abre la publicación en una pestaña nueva.
- ☐ Los procesos sin estos datos se ven bien (sin "undefined" ni huecos raros).

T-17 · Ciclo de vida del contrato

🔵 Junior+ · **L** · 🟠 **Requiere decisión**

Historia de usuario

> Como **director**, quiero marcar un contrato como "En cierre / liquidación" y luego "Cerrado", para separar los contratos que siguen activos de los que solo esperan trámites administrativos.

Contexto

- Requisito RF-24: estados **En ejecución** → **En cierre/liquidación** → **Cerrado**.
- Hoy el contrato solo tiene `activo` (sí o no), que usa el borrado lógico.

🟠 **Decisión necesaria:**

1. ¿Se puede volver de "En cierre" a "En ejecución"? (Propuesta: sí; de "Cerrado" no se vuelve.)
2. ¿Los contratos cerrados cuentan en el dashboard y en las alertas? (Propuesta: no.)

Guía técnica

1. `schema.prisma`: enum `EstadoContrato { EN_EJECUCION EN_CIERRE CERRADO }` y campo `estado EstadoContrato @default(EN_EJECUCION)`. Crea la migración.
2. Backend: endpoint `PATCH /contratos/:id/estado` (SUPERVISOR o superior) que valide las transiciones con una tabla `TRANSICIONES`, igual que en `procesos.service.ts`.
3. Los contratos `CERRADO` se excluyen de los KPIs (`dashboard.service.ts`) y de las alertas (`notificaciones.service.ts`).
4. Frontend: muestra el estado en la lista y en el detalle, agrega botones de cambio de estado y un filtro por estado.

Criterios de aceptación

- ☐ Los contratos existentes quedan en "En ejecución" después de la migración.
- ☐ Solo se permiten las transiciones acordadas; cualquier otra responde **400**.
- ☐ Un contrato cerrado no aparece en las alertas ni suma en el dashboard.
- ☐ COLABORADOR no puede cambiar el estado (**403**) y no ve los botones.
- ☐ El filtro por estado funciona en la lista de contratos.

Fuera del alcance de este backlog (requieren perfil semi-senior o senior)

Estos pendientes del alcance **no** son adecuados para alguien junior, por riesgo, por infraestructura o por decisiones de arquitectura. Quedan anotados para planificarlos aparte.

Pendiente	Por qué no es para junior
Notificaciones push iOS/Android (Capacitor + OneSignal + APNs, RF-30 a RF-37)	Requiere cuenta Apple Developer, macOS con Xcode, certificados y un job programado (cron).
Recuperación de contraseña por email (RF-05)	Requiere servidor de correo (SMTP) on-premise y tokens de un solo uso. Es un tema delicado de seguridad.
Auditoría de todos los cambios de un proceso (RF-11)	Hoy solo se registran los cambios de estado; registrar cada campo exige rediseñar `historial_proceso`.
Roles del alcance (administrador / director / analista / visualizador)	Hoy hay 3 roles distintos. Cambiarlos afecta permisos en todo el sistema y necesita una migración de datos.
Kanban con arrastrar y soltar (RF-12)	Requiere una librería de *drag & drop* y manejo de estados intermedios; hoy funciona con botones.
Migrar la carga de datos a TanStack Query	Refactor transversal de todas las páginas; permite reactivar la regla de lint `react-hooks/set-state-in-effect`.
Despliegue en el servidor on-premise	Infraestructura: HTTPS/TLS, servicio del sistema, backups de PostgreSQL y variables de producción.
Migración de los datos del Excel actual	Requiere conocer bien el Excel fuente y validar datos con el negocio.
Login con Google Workspace / Microsoft 365 (v2.0)	OAuth2/OIDC; arquitectura de autenticación.